



Rapport de Projet UML

Gestion des Rapports Quotidiens de Vol (RQV)

Université Toulouse - L3 MIASHS

MJEKU Olti
SAID Karim
WABOE Josue Hyacinthe

Encadrant :
M. S. EBERSOLD

Janvier 2026

Table des matières

1	Introduction	2
2	Analyse et Conception UML	2
2.1	Acteurs et Cas d'Utilisation	2
2.2	Évolution du Diagramme de Classes	2
3	Mise en œuvre du Patron Observer	4
3.1	Architecture du Patron	4
3.2	Implémentation Java	5
4	Tests et Validation	5
4.1	Stratégie de Tests	5
4.2	Exemple de Test : Génération du Rapport	5
4.3	Trace d'Exécution	6
5	Conclusion	6

1 Introduction

Ce projet a pour objectif la conception et le développement d'une application Java permettant la gestion des déclarations de vols de véhicules (vélos et véhicules à moteur) au sein d'un commissariat. L'application doit permettre aux victimes et témoins de déclarer des vols, et aux agents de police de générer des Rapports Quotidiens de Vol (RQV).

Une contrainte technique majeure de ce projet était la mise en œuvre rigoureuse du patron de conception *Observer* pour gérer le système d'objets trouvés, ainsi qu'une démarche de modélisation UML incrémentale.

2 Analyse et Conception UML

2.1 Acteurs et Cas d'Utilisation

Nous avons identifié trois acteurs principaux interagissant avec le système :

- **La Victime / Le Témoin** : Ils créent des déclarations (véhicule ou vélo) et consultent l'état de leur dossier.
- **L'Agent de Police** : Il s'authentifie, génère les RQV quotidiens et met à jour le statut des déclarations (ex : passage à l'état "Résolue").

2.2 Évolution du Diagramme de Classes

Notre conception a évolué entre la phase d'analyse initiale et la phase finale de développement. Cette itération nous a permis d'améliorer la cohésion de nos classes.

Version 1 : Conception Initiale

Dans notre première version, la classe `Declaration` était très chargée (voir Figure 1).

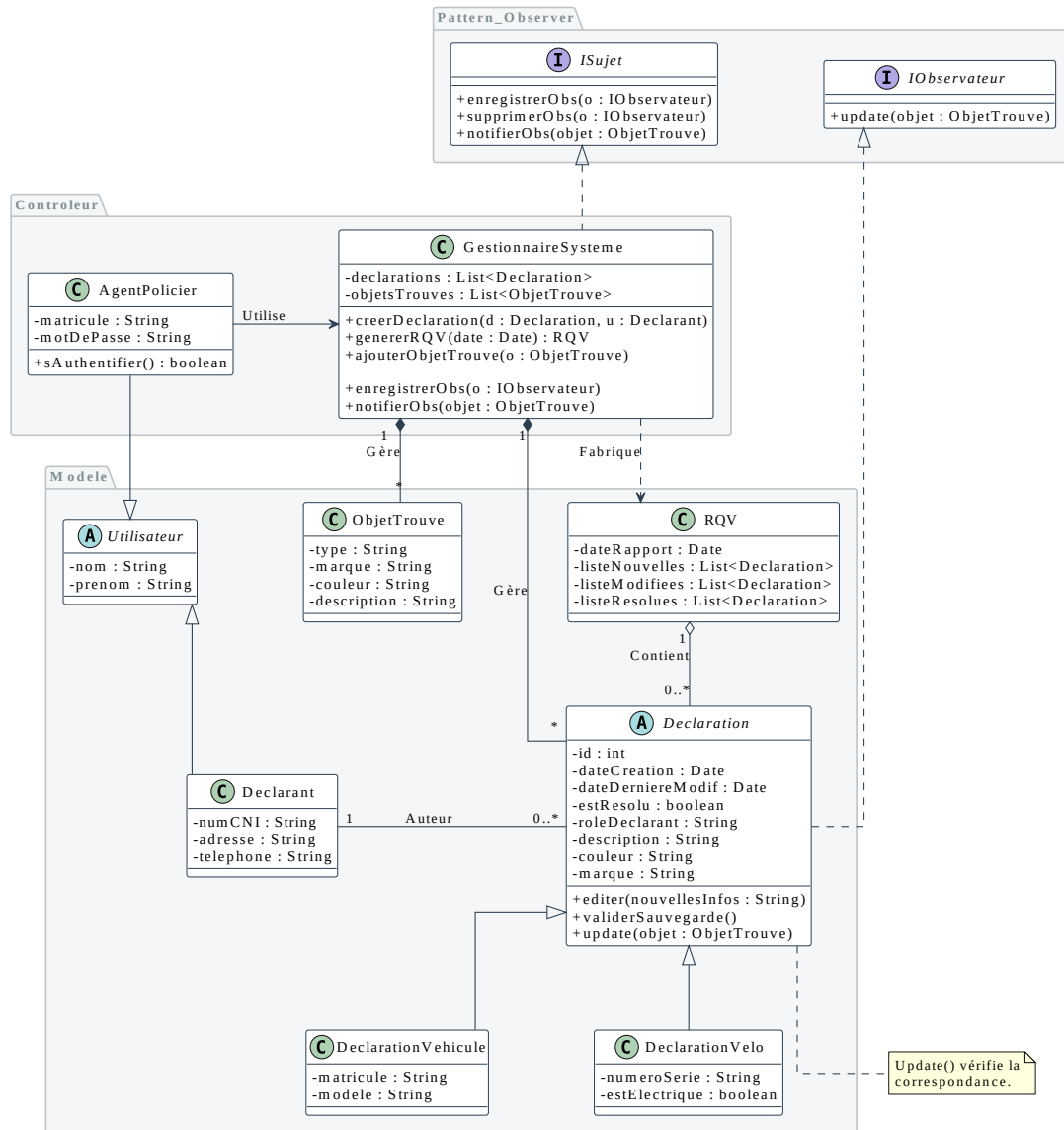


FIGURE 1 – Diagramme de Classe - Version 1

Elle contenait directement tous les attributs liés à l'adresse du vol (rue, ville, code postal), ce qui alourdissait la classe. De plus, la gestion des notifications n'était pas encore modélisée sous forme d'objet distinct.

Version 2 : Conception Finale et Améliorations

Suite à une relecture critique, nous avons produit la version 2 avec les modifications suivantes (voir Figure 2) :

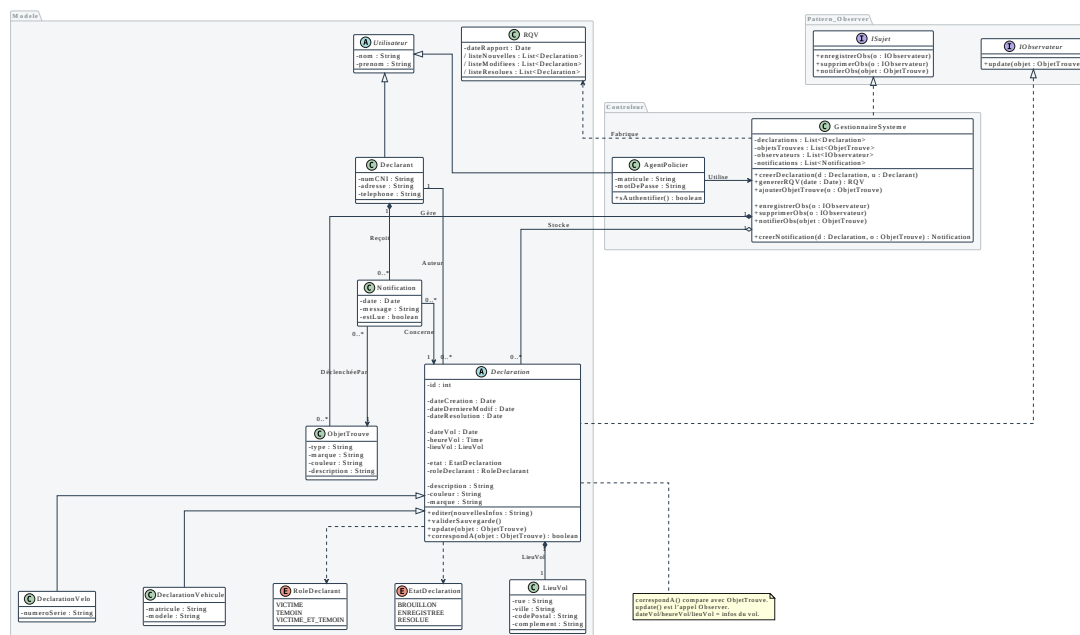


FIGURE 2 – Diagramme de Classe - Version Finale

- **Création de la classe LieuVol** : Nous avons extrait les informations géographiques dans une classe dédiée. Cela allège la classe **Declaration** et respecte le principe de responsabilité unique.
- **Ajout de la classe Notification** : Pour persister les messages envoyés aux utilisateurs (lorsqu'un objet est retrouvé), nous avons ajouté une entité **Notification** liée au **Declarant**.
- **Enums EtatDeclaration et RoleDeclarant** : Afin d'éviter les chaînes de caractères propices aux erreurs ("victim", "Victime", etc.), nous avons typé fortement ces concepts.

3 Mise en œuvre du Patron Observer

Conformément au cahier des charges, nous avons utilisé le patron *Observer* pour le système d'objets trouvés. L'objectif est de notifier automatiquement les déclarants lorsqu'un objet correspondant à leur déclaration est entré dans le système.

3.1 Architecture du Patron

- **Sujet (ISujet)** : C'est notre contrôleur principal, `GestionnaireSysteme`. Il maintient une liste d'observateurs.
- **Observateur (IObservateur)** : C'est l'interface implémentée par nos déclarations.

3.2 Implémentation Java

Lorsqu'un nouvel `ObjetTrouve` est ajouté via le contrôleur, la méthode `notifierObs` est déclenchée. Voici un extrait de notre classe `GestionnaireSysteme` montrant cette logique :

```
1 public void notifierObs(ObjetTrouve objet) {
2     for (IObservateur o : observateurs) {
3         // On caste l'observateur en Declaration pour vérifier la
4         // correspondance
5         Declaration d = (Declaration) o;
6
7         // Si la d clARATION correspond à l'objet trouv (ex: même
8         // marque/couleur)
9         if (d.correspondA(objet)) {
10             o.update(objet); // Notification (création d'alerte, email,
11             // etc.)
12         }
13     }
14 }
```

Listing 1 – Méthode de notification dans `GestionnaireSysteme.java`

La méthode `correspondA()` dans la classe `Declaration` contient la logique métier de comparaison (marques, numéros de série, etc.), ce qui encapsule bien les règles de correspondance.

4 Tests et Validation

4.1 Stratégie de Tests

Nous avons utilisé **JUnit 5** pour valider le fonctionnement de notre application. Nos tests couvrent :

- La création et la validation des déclarations (`DeclarationTest.java`).
- Le cycle de vie d'un RQV (`RQVTest.java`).
- La logique de notification du pattern Observer.

4.2 Exemple de Test : Génération du Rapport

Un point critique était de s'assurer que le rapport affiche les bonnes statistiques. Dans `RQVTest.java`, nous avons simulé des flux d'entrées/sorties pour vérifier l'affichage console exact :

```
1 @Test
2 void afficherRapport_ecrit_un_resume_avec_les_bonnes_valeurs() {
3     RQV rqv = new RQV(new Date());
4     // ... Ajout de d clARATIONS ...
5
6     // Capture de la sortie console pour vérification
7     ByteArrayOutputStream out = new ByteArrayOutputStream();
8     System.setOut(new PrintStream(out));
9
10    rqv.afficherRapport(); // Ex cUTION
11
12    String s = out.toString();
13    assertTrue(s.contains("Nouvelles d clARATIONS: 1"));
14    assertTrue(s.contains("D clARATIONS modifi es: 0"));
15 }
```

Listing 2 – Extrait de `RQVTest.java`

4.3 Trace d'Exécution

Voici la trace d'exécution obtenue lors du lancement du scénario principal (`Main.java`) qui simule une journée type :

```
--- DÉMARRAGE SYSTÈME GESTION RQV ---
[INFO] Chargement des données... OK.
[AUTH] Agent de police 'Matricule-789' connecté avec succès.

> Création déclaration #1 : Vol Véhicule (Peugeot 208, Noir) - VICTIME
> Création déclaration #2 : Vol Vélo (Decathlon, Rouge) - TEMOIN

[ACTION] Ajout d'un objet trouvé : Vélo Decathlon Rouge
[OBSERVER] Correspondance trouvée pour la déclaration #2 !
[NOTIFICATION] Le déclarant (Témoin) a été notifié.

[RQV] Génération du rapport quotidien pour le 15/01/2026...
-----
          RAPPORT QUOTIDIEN DE VEILLE (RQV)
          Date : 15/01/2026
-----
- Nouvelles déclarations : 2
- Déclarations modifiées : 0
- Déclarations résolues  : 0
-----
[INFO] Rapport sauvegardé.
```

5 Conclusion

Ce projet nous a permis de confronter la théorie de la conception UML à la pratique du développement Java. L'utilisation du patron *Observer* s'est révélée très pertinente pour découpler la gestion des objets trouvés des déclarations spécifiques. La transition entre notre diagramme de classe V1 et V2 montre également l'importance d'une phase de refactoring pour assurer la maintenabilité du code.